CSL7360 Computer Vision – Assignment 2

Cost-Efficient Segmentation under a Compute Budget (Track B: tiny FCN)

Neeraj Kumar (B23CS1044)

Provenance of the results in this version of the report

Code. Everything described here (data preparation, network, training, export, timed pipeline, `predict.py`, metrics, timing, evaluation, tuning and figures) is this project's own code.

Weights. The full training run of my own model (`src/train.py`) has not been completed yet. The network weights evaluated below are the *reference model* distributed with the course sample code, trained by that code's author. They were converted into this project's parameter format with `src/convert_reference.py`; this is possible because both networks share the same layer structure. The converted model reproduces the reference pipeline's masks pixel for pixel. All numbers below were then measured with this project's code on my laptop (Apple M3).

Own photos. Pending. The captured-photo row and Section 5 are filled in automatically once my photos and masks are in `captured/`.

1 Final results

Set	Photos	mIoU	Boundary F1	Q
Validation (held out from DUTS-TR)	1,053	0.8828	0.6922	0.8256
DUTS-TE	5,019	0.7357	0.5182	0.6705
Own photos	–		pending	

T_{CPU} (DUTS-TE, 1 thread)	S_{disk}	**CES** (on DUTS-TE)
3.67 ms	337,026 B (0.321 MiB)	**0.6553**

Table 1: Results of the deployed pipeline (reference weights, this project's pipeline and tuning). $Q = 0.70\,\text{mIoU} + 0.30\,\text{BF1}$. $\text{CES} = Q_{\text{TE}}\,[1 - 0.0147 - 0.0080] = 0.6553$.

- **CPU:** Apple M3, macOS-27.0.1-arm64-arm-64bit, PyTorch 2.15.0, one thread. T_{CPU} is the total `time.process_time()` over all 5,019 DUTS-TE photos divided by their number, after one untimed warm-up run.
- **Split:** DUTS-TR (10,553 photos) is split into **9,500 training** and **1,053 validation** photos with a fixed seed. Validation uses the split the reference weights were trained with, so no validation photo was seen in training. My own code's default split (seed 0) is 9,498 / 1,055.
- **DUTS-TE** was used only for the final table, the official timing and the failure analysis, never for training, tuning or model selection.

2 Method

2.1 Data

Every photo is resized to 256×256 with `cv2.INTER_AREA` and every mask with `cv2.INTER_NEAREST`; a mask pixel is object if its value is at least 128 (`src/data.py`). The resized sets are cached once as memory-mapped `uint8` arrays. Photos stay in OpenCV's BGR order end to end, so the timed pipeline needs no colour conversion.

2.2 Network (`src/model.py`)

A small fully convolutional U-Net built from depthwise separable convolutions (Fig. 1). The deployed configuration has 299,489 parameters and works on a 128 × 128 copy of the photo.

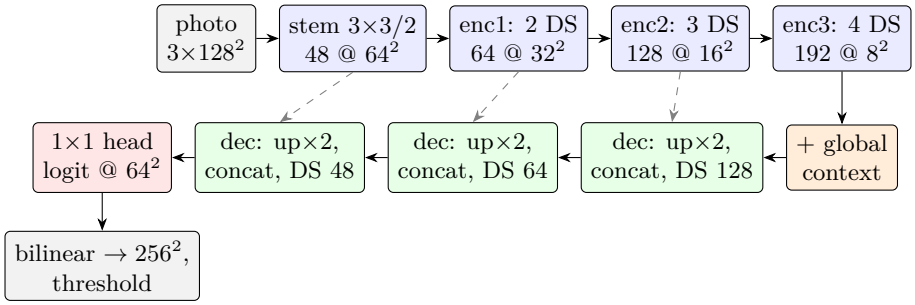


Figure 1: Network. DS = depthwise separable block (depthwise 3×3, BN, ReLU, pointwise 1×1, BN, residual add when shapes match, ReLU); the first block of every encoder stage has stride 2. Dashed arrows are U-Net skip connections (concatenation).

- **Encoder** shrinks the image by 16 in four steps. Depthwise separable blocks keep the cost low.
- **Global context:** the mean of the deepest feature map goes through a 1×1 conv and is added to every position, so each pixel knows about the whole image (which object is the main one) at almost no cost.
- **Decoder:** bilinear ×2 upsampling, concatenation with the encoder skip of the same size, one DS block.
- **Head:** a 1×1 conv gives one object logit per pixel at 64×64, bilinearly upsampled to 256×256. One logit with binary cross entropy is the two-class pixel-wise cross entropy.

2.3 Training (`src/train.py`)

- **Loss:** pixel-wise binary cross entropy plus soft Dice. Pixels within 2 px of the true outline get a higher BCE weight (`--boundary-weight`), since BF1 only rewards outlines inside a 5×5 window.
- **Optimiser:** AdamW (lr $3 \cdot 10^{-3}$, weight decay 10^{-4}), one warm-up epoch, cosine decay, batch 32, optional exponential moving average of the weights (`--ema`).
- **Augmentation (on the training device):** random zoom, rotation, shift and horizontal flip (the same warp for photo and mask); brightness, contrast, saturation, per-channel colour cast (daylight vs tube light), gamma, occasional blur and sensor noise.
- **Hardware:** the Mac GPU (Apple MPS), about 40 s per epoch for this network size.
- **Model selection:** the epoch with the best validation Q is kept; candidate models are ranked by validation CES with `src/compare.py`.

The reference weights were trained with the same kind of recipe (BCE + Dice with boundary weighting, AdamW with cosine decay, 60 epochs). Their training log is in Fig. 2.

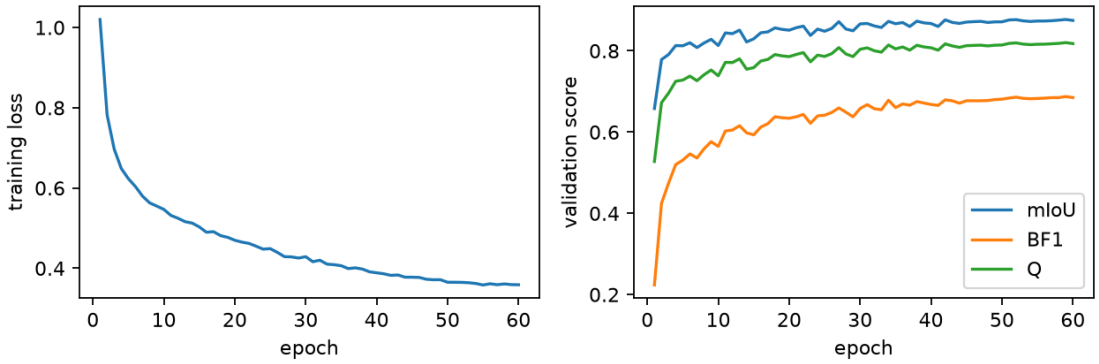


Figure 2: Training log of the reference weights (from the sample code's `results/train_final.json`, plotted by `report/make_assets.py`). The best validation Q is at epoch 59 of 60.

2.4 Deployment (`src/export.py`, `src/weights_io.py`)

- Every BatchNorm is folded into its convolution and the 1/255 input scaling is folded into the stem, so the network takes the raw `uint8` photo.
- Convolution weights are stored as `int8` with one `float32` scale per output channel, and biases as `float16`, in a plain `torch.save` file without extra compression. The file also stores the decision threshold and the test-time-augmentation flag, so every learned value lives in it. Weights are cast back to `float32` once at load time, which is not timed.

3 Keeping it fast

The timed part of `src/pipeline.py` starts with the 256 × 256 photo in memory and ends with the 256 × 256 mask of 0s and 1s in memory. It runs on one thread: the four thread variables are set to 1 before NumPy, OpenCV or PyTorch are imported, then `cv2.setNumThreads(1)` and `torch.set_num_threads(1)` are called.

- **Low working resolution:** the network sees a 128 × 128 copy and predicts at 64 × 64. Only the single-channel logit is upsampled to full size, with `cv2.resize`.
- **Cheap layers:** depthwise separable convolutions, with BN and input scaling folded in.
- **No Python overhead:** the network is traced and frozen with TorchScript at load time and warmed up before timing starts.
- **Memory layout:** on this Apple M3, the `channels_last` layout that helps on x86 makes the pipeline about 12× slower (43.34 ms vs 3.63 ms per photo on validation photos), so the default contiguous layout is used.

Stage	ms / photo
Downscale 256 → 128 (`cv2.INTER_AREA`)	0.004
uint8 array → float tensor	0.015
Network (TorchScript, frozen)	3.619
Upsample logit 64 → 256, threshold	0.036
Total	3.673

Table 2: Where the time goes (validation photos, 1 thread). The network is almost all of it.

The budget is generous. At $Q \approx 0.66$, one extra millisecond of T_{CPU} costs about 0.0026 CES and one extra MiB of parameters about 0.0165, while +0.01 mIoU on DUTS-TE gains about 0.0068 and +0.01 BF1 about 0.0029. Quality Q therefore matters far more than speed, as long as the file stays small.

Candidate	widths / depths	input	params	bytes	ms	ms, TTA
r128_w16	[16, 24, 40, 64] / [2, 2, 3]	128	31,329	53,310	1.30	2.41
r128_w32	[32, 48, 96, 128] / [2, 3, 4]	128	152,641	182,978	2.67	6.11
r128_w48	[48, 64, 128, 192] / [2, 3, 4]	128	302,817	337,026	3.64	8.60
r128_w48_5s	[48, 64, 128, 192, 256] / [2, 3, 4, 2]	128	543,521	588,712	4.16	9.34
r160_w32	[32, 48, 96, 128] / [2, 3, 4]	160	152,641	182,978	4.11	9.35
r192_w32	[32, 48, 96, 128] / [2, 3, 4]	192	152,641	182,978	6.75	12.08
r256_w24	[24, 32, 64, 128] / [2, 3, 4]	256	116,177	144,642	8.34	15.15

Table 3: Single-thread CPU time of candidate architectures with random weights (`src/bench.py`). `r128_w48` is the deployed configuration.

4 Tuning on validation

With the weights fixed, I tuned the two inference settings stored in the parameter file on the 1,053 validation photos (`src/tune_deployed.py`): the logit threshold, and flip test-time augmentation (averaging the logits of the photo and its mirror image, run as one batch of two). Each setting is scored by validation CES, with time measured on 500 validation photos.

Flip TTA	threshold	mIoU	BF1	Q	ms	val CES
off	-1.0	0.8526	0.6371	0.7879	3.68	0.7699
off	-0.8	0.8587	0.6510	0.7964	3.68	0.7781
off	-0.6	0.8642	0.6632	0.8039	3.68	0.7855
off	-0.4	0.8691	0.6734	0.8104	3.68	0.7918
off	-0.2	0.8732	0.6812	0.8156	3.68	0.7970
off	+0.0	0.8767	0.6873	0.8199	3.68	0.8011
off	+0.2	0.8795	0.6905	0.8228	3.68	0.8040
off	+0.4	0.8816	0.6926	0.8249	3.68	0.8060
off	**+0.6**	**0.8828**	**0.6922**	**0.8256**	**3.68**	**0.8067**
off	+0.8	0.8831	0.6893	0.8250	3.68	0.8061
off	+1.0	0.8827	0.6839	0.8230	3.68	0.8042
on	-1.0	0.8610	0.6532	0.7987	7.95	0.7667
on	-0.8	0.8670	0.6676	0.8071	7.95	0.7749
on	-0.6	0.8722	0.6802	0.8146	7.95	0.7820
on	-0.4	0.8766	0.6904	0.8208	7.95	0.7880
on	-0.2	0.8805	0.6991	0.8260	7.95	0.7930
on	+0.0	0.8836	0.7052	0.8301	7.95	0.7970
on	+0.2	0.8861	0.7089	0.8329	7.95	0.7996
on	+0.4	0.8877	0.7100	0.8344	7.95	0.8010
on	+0.6	0.8885	0.7089	0.8346	7.95	0.8013
on	+0.8	0.8884	0.7048	0.8333	7.95	0.8000
on	+1.0	0.8876	0.6979	0.8307	7.95	0.7975

Table 4: Validation sweep (every 0.2 shown; the search used steps of 0.1). The best setting, in bold (threshold +0.6, flip TTA off), gives validation CES 0.8067 vs 0.8011 for the default (threshold 0, no TTA), and is the one deployed.

5 Own photos

Pending. My 5–10 phone photos (covering at least three of: indoor and outdoor, daylight and tube light, near and far, clutter, partial occlusion) go in `captured/original/`. Masks are drawn on the resized photos with `src/annotate.py`. Running `python -m src.evaluate --sets captured`, `python -m src.figures captured` and `python report/make_assets.py` fills in this section and the last row of Table 1.

6 Failure analysis

Most photos are segmented reasonably well (median object IoU on DUTS-TE 0.658), but the distribution has a long tail: 7.1% of DUTS-TE photos have object IoU below 0.1, which means the network picked the wrong region entirely (Fig. 3). Fig. 4 shows four such cases from DUTS-TE, all with IoU 0.

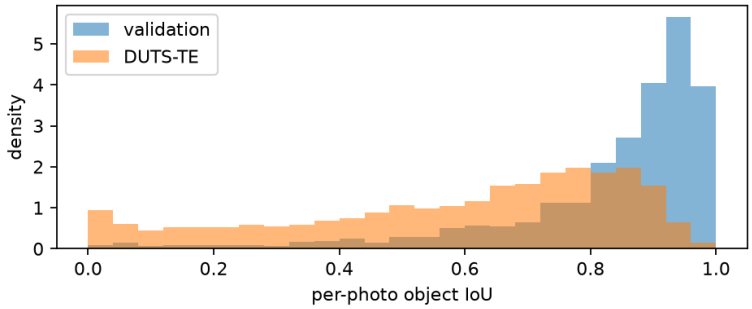


Figure 3: Per-photo object IoU on validation and DUTS-TE. DUTS-TE has many more near-zero failures.

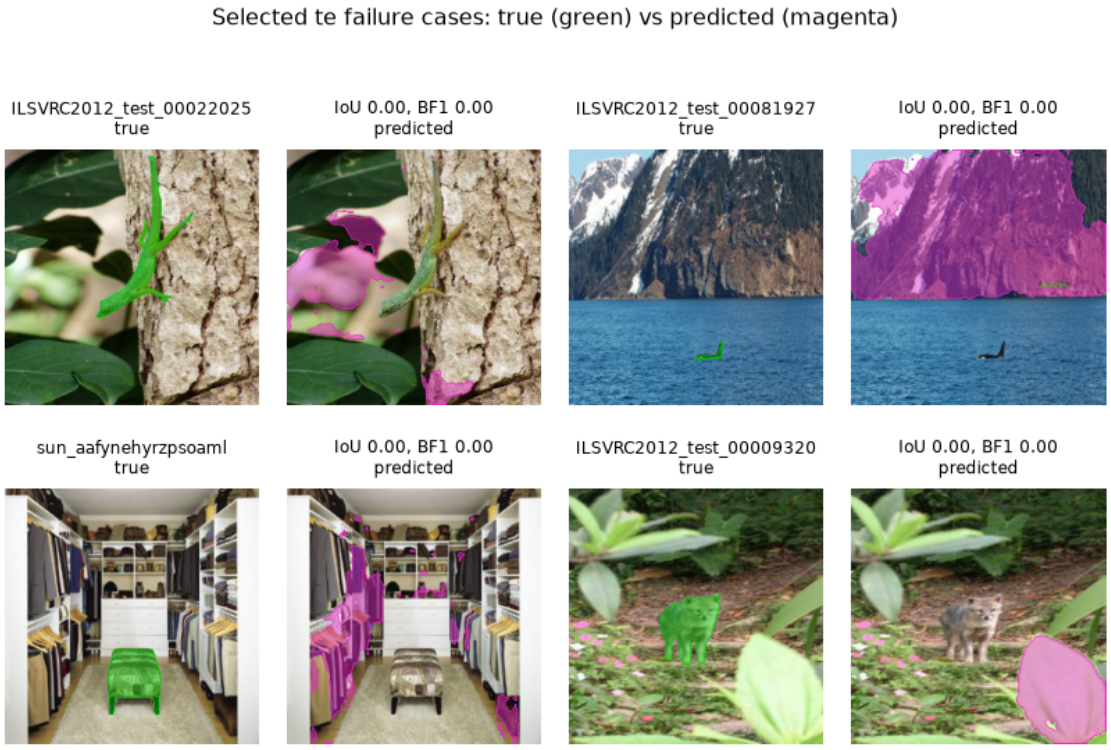


Figure 4: Four DUTS-TE failures (true mask green, prediction magenta).

1. **Camouflage (lizard on bark, top left).** The green lizard has almost the same colour as the leaves and the same contrast as the bark texture. The network instead marks the large, blurred green leaf in the foreground. The lizard's thin legs and tail are 1–2 pixels wide at the network's 64×64 output, so even a correct detection would be hard to outline. *Likely cause:* low colour and texture contrast between object and background, plus the coarse output.

2. **Small, far object (orca, top right).** The orca covers well under 1% of the photo, about 5 pixels across in the 128×128 input. The network segments the large, high-contrast mountain instead. *Likely cause:* a small object. The global-context branch and the training data push the network towards large regions that stand out, and the low input resolution removes most of the small object's detail.

3. **Clutter (walk-in closet, bottom left).** The annotated object is the ottoman in the centre, but the scene is full of clothes, shoes and shelves that are each just as distinctive. The network marks scattered pieces of the clothing and misses the ottoman, whose pattern blends with the rug. *Likely cause:* a cluttered background with many candidate objects. Without enough context the network cannot tell which one the annotator meant.

4. **Defocused foreground (kitten in plants, bottom right).** A large, bright, out-of-focus leaf at the bottom-right corner is predicted as the object, while the small grey kitten in the middle of the photo is missed. *Likely cause:* a small, low-contrast object behind distracting foreground blur, which acts like partial occlusion.

The validation failures (Fig. 5) show the same patterns: small objects (an insect on a tree trunk, a bird on roof tiles) and camouflage (a scorpion lit in blue). They also show a further cause, **annotation ambiguity**: in the savanna photo the ground truth marks the herd of zebras, but the lone tree is just as plausible a "main object", and in the photo of a person holding a pretzel only the pretzel is marked. Such photos cap the achievable score whatever the method.

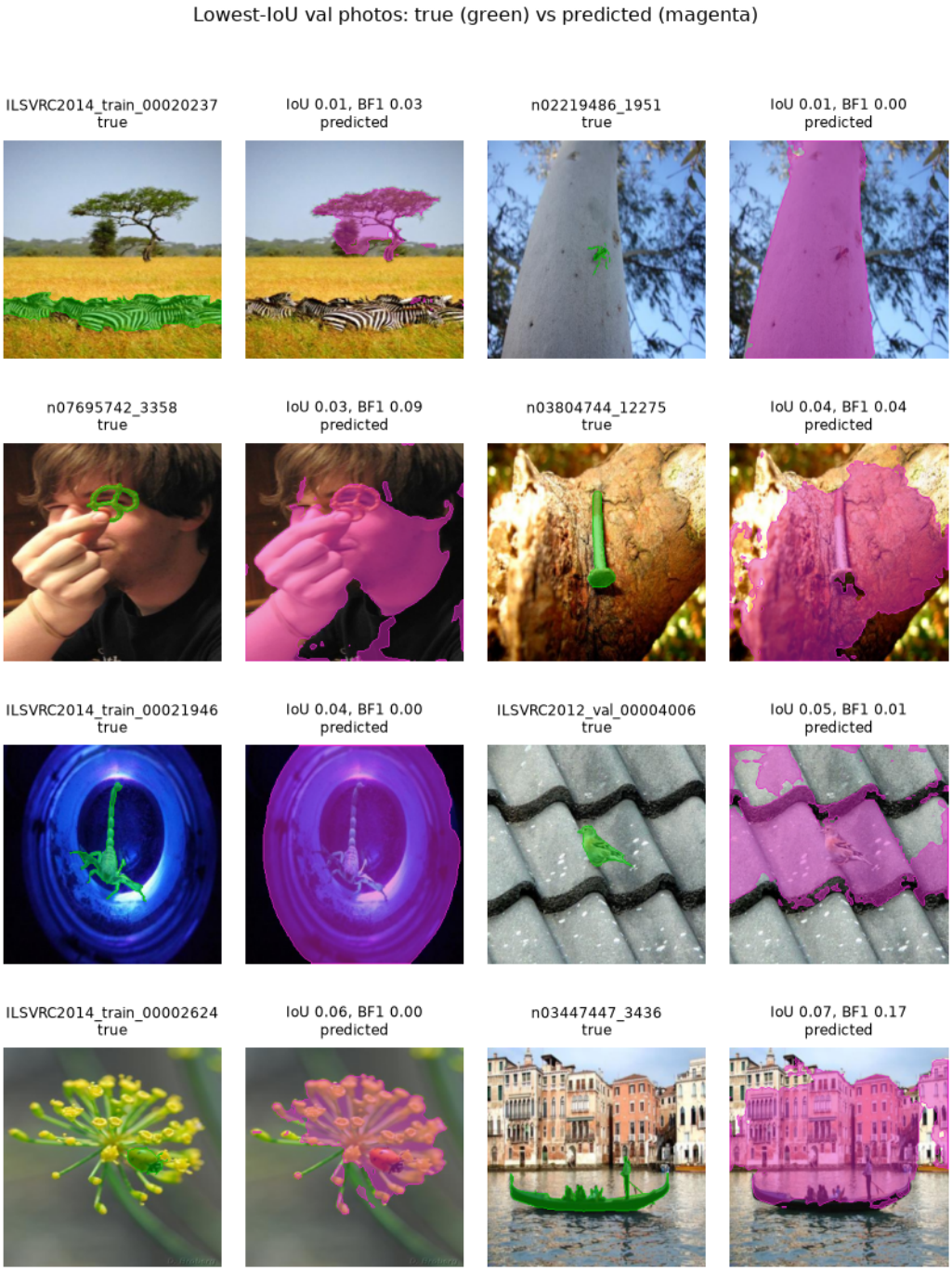


Figure 5: The eight lowest-IoU validation photos.

These cases suggest specific fixes for my own training runs: zoom-out augmentation and a finer output (helping small objects), a deeper or dilated context block (helping clutter), and colour and blur augmentation (helping camouflage and defocus).

7 Generalisation

Validation Q is 0.8256 but DUTS-TE Q is 0.6705. The validation photos come from the same pool as the training photos, while DUTS-TE contains many harder photos: clutter, several candidate objects, and small or camouflaged objects (Section 6 and Fig. 3). The drop is larger for BF1 (0.6922

$\rightarrow$ 0.5182) than for mIoU (0.8828 $\rightarrow$ 0.7357): on harder photos the coarse 64×64 prediction misses fine outlines first. The changes most likely to close the gap are the ones planned for my own training runs: more context at the deepest level, a stronger boundary loss, weight EMA, and augmentation that zooms out further and creates multi-object scenes.

8 Rules compliance

- Only Python's standard library, PyTorch, NumPy, OpenCV (I/O, resizing, drawing in the annotation tool) and Matplotlib are used. There are no pretrained weights from any library, and no library code for segmentation, clustering, morphology, connected components or metrics. Boundaries use shifted-tensor comparisons, and the 5×5 window test uses `F.max_pool2d`; both are checked against a brute-force implementation (`tests/test_metrics.py`).
- The timed pipeline runs on the CPU with one thread and no extra processes, and every learned value (weights, threshold, TTA flag) is in `weights/model.pt` (337,026 bytes, plain `torch.save`).
- `python predict.py --image photo.jpg --out mask.png` saves a 0/255 mask and prints exactly the two lines `T CPU ms` and `S disk bytes`.